Comparison of algorithms for estimating regression splines using lasso penalty

Eun-Ji Lee & Jong-Beom Park & Dong-Young Lee & Jae-Hwan Jhong*

Department of Information Statistics, Chungbuk National University

Introduction

- With the rise of big data, the rapid increase in data volume and variety makes true function estimation challenging.
- Lasso (Least Absolute Shrinkage and Selection Operator) regression (Tibshirani, 1996) is a key method for identifying influential variables and their impacts.
- Lasso regression incorporates a penalty term into the residual sum of squares, enabling variable selection.
- In truncated power spline regression, applying an l_1-norm penalty on basis parameters can be viewed as a knot selection problem.
- This approach helps select significant knots from candidates within the predictor's range, similar to variable selection in response modeling.

Lasso Regression Splines Model

- The Lasso (Least Absolute Shrinkage and Selection Operator) regression model is a popular penalization technique for selecting important variables in high-dimensional data.
- In regression splines, Lasso regression can address the knot selection problem by applying an l_1-norm penalty to the parameters associated with the spline basis.

For $i = 1, \ldots, n$, consider the multiple regression model:

$$y_i = \sum_{j=1}^{p} \delta_j x_{ij} + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma^2) \tag{1}$$

- Here, y_i is the response variable, x_{ij} represents the j-th predictor for the i-th sample, and δ_j are the parameters.
- To perform variable selection, the Lasso estimator minimizes the residual sum of squares with an l_1-norm penalty:

$$\hat{\delta}_{\text{lasso}} = \arg\min_{\delta \in \mathbb{R}^p} \sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{p} \delta_j x_{ij} \right)^2 \quad \text{subject to} \quad \sum_{j=1}^{p} |\delta_j| \le t \tag{2}$$

- Here, $t > 0$ is the tuning parameter controlling the sparsity of the model.
- In spline regression models, these penalties can help select significant knots among candidates, allowing efficient model fitting even for complex data structures.

Methods

To solve the Lasso regression splines model, we apply three main algorithms: Alternating Direction Method of Multipliers (ADMM), Coordinate Descent Algorithm (CDA), and Quadratic Programming (QP). Each algorithm optimizes the Lasso objective under different methods.

- Alternating Direction Method of Multipliers (ADMM):
  - Minimizes the augmented Lagrangian:

$$L(\alpha, \beta, \gamma, u) = \frac{1}{2n}\|y - X\alpha - P\beta\|_2^2 + \lambda\|\gamma\|_1 + \frac{\xi}{2}\|\beta - \gamma + u\|_2^2 \tag{3}$$

  - Iteratively updates parameters α, β, and γ until convergence.
- Coordinate Descent Algorithm (CDA):
  - Optimizes each coefficient θ_ℓ individually:

$$\theta_\ell \leftarrow \arg\min_{\theta_\ell \in \mathbb{R}} R_\lambda(\theta_1, \ldots, \theta_\ell, \ldots, \theta_{p+K}) \tag{4}$$

  - Cycles through all coefficients until the objective function converges.
- Quadratic Programming (QP):
  - Reformulates the problem as a quadratic program with non-negativity constraints:

$$\min_b \left(-\frac{1}{n} y^\top Z b + \frac{1}{2n} b^\top Z^\top Z b \right) \quad \text{subject to} \quad \sum_{k=1}^{K} \left((\beta_k)^+ + (\beta_k)^- \right) \le t, \tag{5}$$

  - Solves precisely without iterative updates, using the `solve.QP` function.

Each method was evaluated for fitting accuracy and computational efficiency, allowing for algorithm recommendations based on performance in different data scenarios.

Simulation (1)

Example 4: $$f(x) = (x(1-x))^{1/2} \sin\left(\frac{2\pi(1+\tau)}{x+\tau} \right), \quad \tau = 0.05 \tag{6}$$

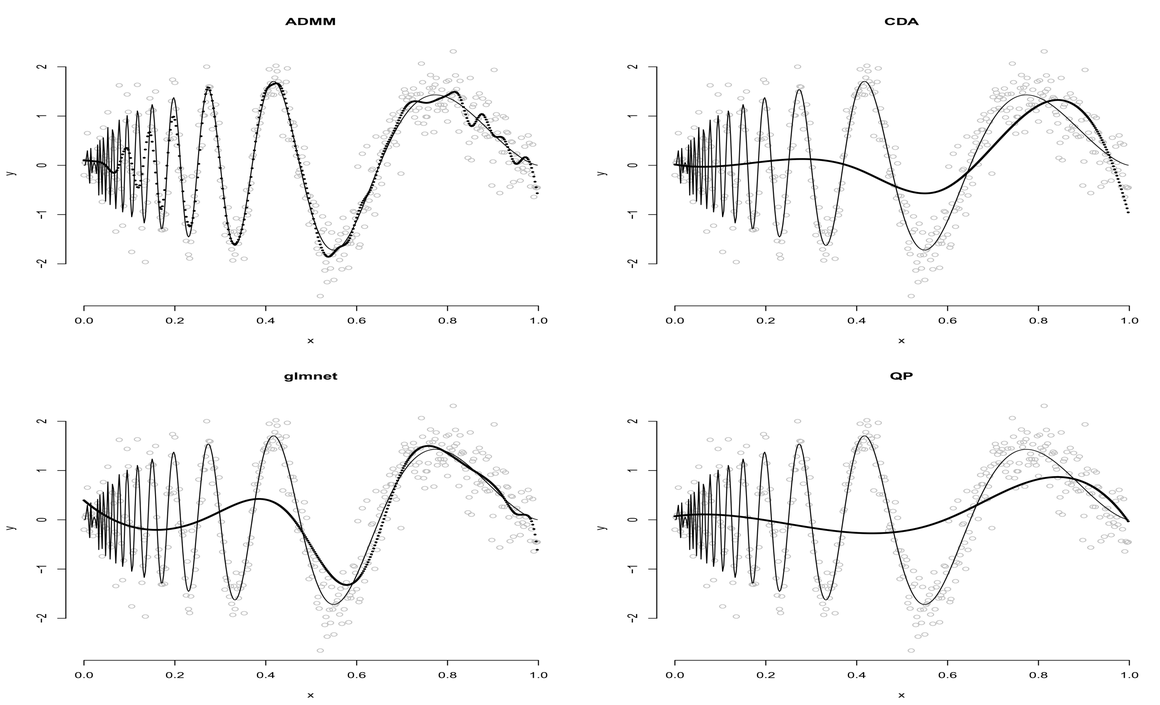

Figure 1: Fitted plots of Example 4 using ADMM, CDA, `glmnet`, QP

Table 1: The average of each criterion over 100 runs of ADMM, CDA, glmnet, and QP for Example 4 with the standard error in parentheses and a sample size $n = 200, 400$ and $\sigma = 0.05, 0.4$. The unit of running time is seconds.

n	σ	Evaluation	ADMM	CDA	glmnet	QP
200	0.05	MSE	1.408×10^{-1}	4.572×10^{-1}	4.694×10^{-1}	7.916×10^{-1}
			(1.605×10^{-4})	(1.452×10^{-3})	(9.089×10^{-3})	(1.308×10^{-3})
		MAE	1.863×10^{-1}	4.431×10^{-1}	4.606×10^{-1}	7.200×10^{-1}
			(1.559×10^{-3})	(8.955×10^{-4})	(6.040×10^{-3})	(1.402×10^{-3})
		MXDV	1.588	1.997	1.957	1.958
			(1.481×10^{-2})	(9.172×10^{-3})	(2.232×10^{-2})	(5.708×10^{-3})
		running time (s)	17.109	17562.529	49.381	2.590
	0.40	MSE	1.582×10^{-1}	4.919×10^{-1}	4.905×10^{-1}	8.071×10^{-1}
			(5.170×10^{-3})	(3.887×10^{-2})	(1.671×10^{-2})	(2.075×10^{-2})
		MAE	2.480×10^{-1}	4.900×10^{-1}	4.9046×10^{-1}	7.313×10^{-1}
			(1.200×10^{-2})	(4.009×10^{-2})	(1.632×10^{-2})	(1.658×10^{-2})
		MXDV	1.594	1.979	1.960	1.941
			(9.123×10^{-2})	(7.078×10^{-2})	(7.410×10^{-2})	(4.811×10^{-2})
		running time (s)	17.135	17706.054	40.187	2.564
400	0.05	MSE	8.010×10^{-2}	4.584×10^{-1}	4.795×10^{-1}	7.870×10^{-1}
			(6.071×10^{-5})	(3.130×10^{-3})	(2.340×10^{-2})	(7.067×10^{-3})
		MAE	1.184×10^{-1}	4.435×10^{-1}	4.765×10^{-1}	7.209×10^{-1}
			(1.367×10^{-3})	(2.097×10^{-3})	(1.610×10^{-2})	(6.422×10^{-3})
		MXDV	1.456	2.006	1.931	1.961
			(1.650×10^{-2})	(1.194×10^{-2})	(4.342×10^{-2})	(1.052×10^{-2})
		running time (s)	34.909	42723.804	65.845	11.769
	0.40	MSE	9.635×10^{-2}	5.616×10^{-1}	5.141×10^{-1}	7.875×10^{-1}
			(3.849×10^{-3})	(5.499×10^{-2})	(2.463×10^{-2})	(6.964×10^{-3})
		MAE	1.879×10^{-1}	5.531×10^{-1}	5.127×10^{-1}	7.211×10^{-1}
			(1.123×10^{-2})	(5.936×10^{-2})	(2.102×10^{-2})	(8.248×10^{-3})
		MXDV	1.533	1.966	1.940	1.964
			(9.103×10^{-2})	(9.155×10^{-2})	(6.290×10^{-2})	(3.708×10^{-2})
		running time (s)	34.829	42749.774	64.719	11.504

Simulation (2)

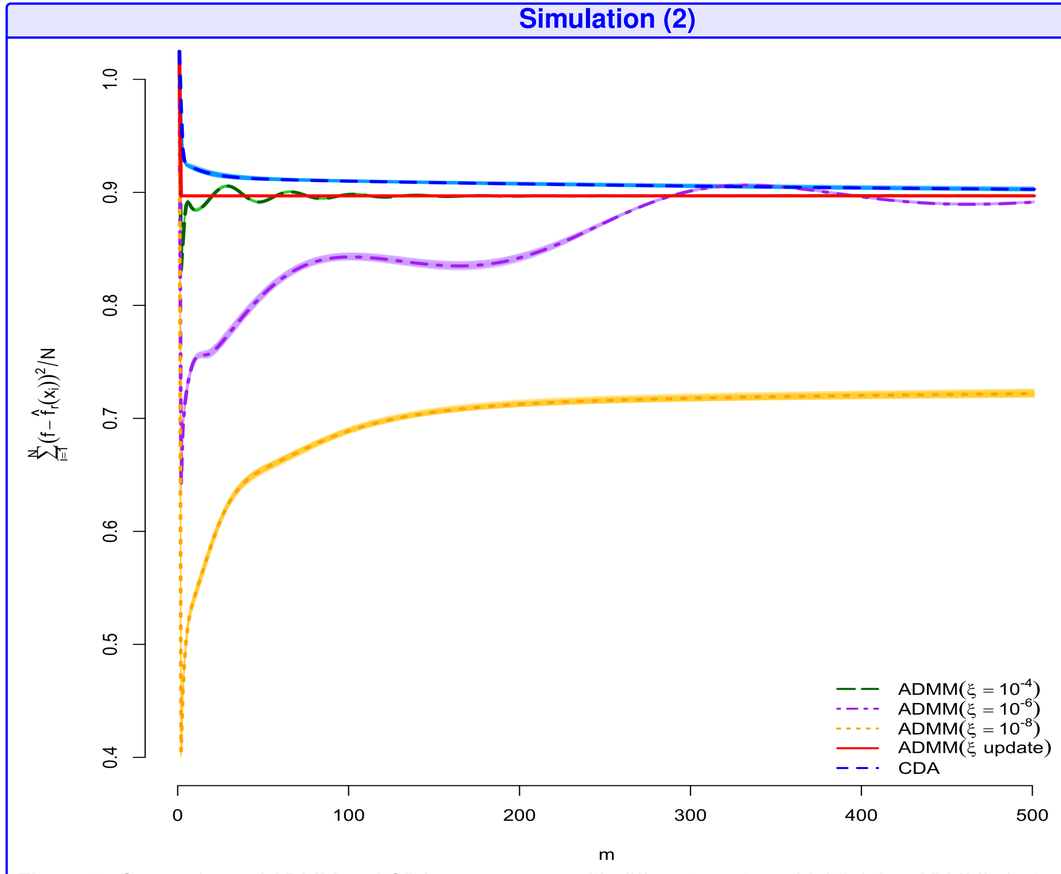

Figure 2: Comparison of ADMM and CDA convergence with different ξ values, highlighting ADMM's faster convergence.

- ADMM converges faster than CDA, especially with the ξ update.
- With the stopping rule, ADMM exits after only 2–3 iterations, while CDA decreases monotonically over more iterations.
- Lower ξ values ($\xi = 10^{-8}$) initially lead to overfitting, followed by gradual convergence.
- ADMM (with $\xi = 10^{-4}$) converges around the 200th iteration; smaller ξ values require more iterations.
- The convergence rate of ADMM is highly sensitive to the ξ value.

Real Data Analysis

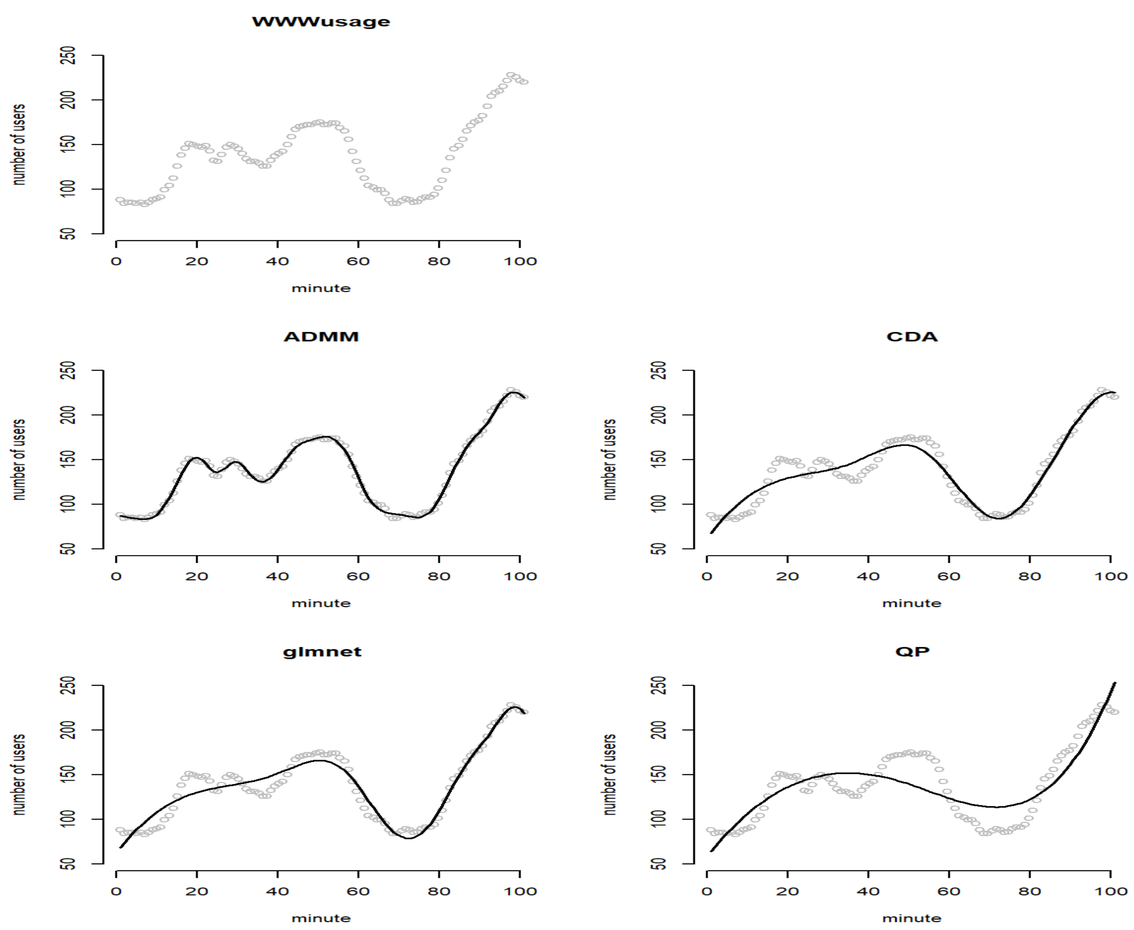

Figure 3: Fitted plots of WWWusage data using ADMM, CDA, `glmnet`, QP

Conclusions

- For simpler forms (constant and linear functions):
  - Fitting performance across the four algorithms was similar.
  - In terms of running time, `glmnet` performed the best.
- For a simple cubic function:
  - `glmnet` and QP did not fit as well as ADMM or CDA.
  - In terms of running time, CDA took approximately 3,000 times longer than ADMM.
- For a complex cubic function:
  - As the number of knots increased, ADMM performed relatively well in terms of both fitting and running time compared to other algorithms.
- Simulation findings:
  - The CDA algorithm has a slow convergence rate for data with complex nonlinear shapes.
  - This issue is attributed to overlapping support when using a truncated power spline as the basis function.
- Recommendations for algorithm selection:
  - Use `glmnet` for simpler forms.
  - Use ADMM for more complex forms.

Acknowledgements

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT)(RS-2024-00342014, RS-2024-00440787, RS-2024-00406939).